

Chapitre 8 : La récursivité

Définition

- Méthode de programmation puissante qui permet d'exprimer d'une manière élégante la solution de problèmes difficiles
- Une fonction s'appelle elle-même (direct)
 - Une condition d'arrêt
 - Un appel avec une complexité diminuée
- Fonctionnement: La pile des appels
- Trouver une solution à un problème en résolvant le même problème mais dans une situation plus simple
- Ce processus de simplification se poursuit **jusqu'à l'atteinte d'un cas où la solution** est connue

Questions clé de la récursivité

Comment exprimer la solution du problème en terme de la solution du même problème p
cas plus simple?

pourquoi chaque appel récursif diminue-t-il la taille du problème?

Quels cas du problème ont des solutions simples et serviront de cas de base?

Les appels récursifs convergent-ils vers des cas de base?

Etude d'un exemple : la fonction factorielle

Pb : Calcul de la factorielle d'un nombre entier n :

On peut définir le factoriel d'un nombre n non négatif de 2 manières.

<i>Définition itérative :</i> $n! = n * n-1 * \dots 2 * 1$ <i>Solution itérative :</i> Fonction FACT (n : entier) : entier VAR i, F: entier <u>DEBUT</u> <u>Si</u> (n = 0) <u>alors</u> FACT $\leftarrow$ 1 <u>Sinon</u> F $\leftarrow$ 1	<i>Définition récursive :</i> $n! = n * (n - 1)!$ et $0! = 1$ <i>Solution récursive :</i> Fonction FACT (n : entier) : entier VAR F : entier <u>DEBUT</u> <u>Si</u> (n = 0) <u>alors</u> FACT $\leftarrow$ 1 <u>Sinon</u> F $\leftarrow$ n * FACT (n-1)
<u>Pour</u> i de n à 1 (pas = -1) faire F $\leftarrow$ F * i <u>Finpour</u> FACT $\leftarrow$ F <u>Finsi</u> <u>FIN</u>	<u>Finsi</u> FACT $\leftarrow$ F <u>FIN</u>

Etude d'un exemple : la fonction factorielle

Mécanisme de fonctionnement de la récursivité

Quand on calcule la factorielle d'un nombre à l'aide d'une fonction récursive, l'exécution suit une série d'appels successifs.

Prenons l'exemple de la factorielle de 4 :

- La fonction `fact(4)` attend le résultat de `fact(3)`
- Celle-ci appelle à son tour `fact(2)`
- Puis `fact(1)`
- Enfin `fact(0)`

À ce stade, la condition d'arrêt est atteinte (puisque la factorielle de 0 vaut 1).

La valeur obtenue est alors renvoyée étape par étape :

- de `fact(0)` vers `fact(1)`
- puis vers `fact(2)`
- jusqu'à `fact(4)`

```
Facto(4) renvoie 4 * Facto(3)
  Facto(3) renvoie 3 * Facto(2)
    Facto(2) renvoie 2 * Facto(1)
      Facto(1) renvoie 1 * Facto(0)
        Facto(0) renvoie 1 (arrêt de la récursion)
      Facto(1) renvoie 1 * 1 = 1
    Facto(2) renvoie 2 * 1 = 2
  Facto(3) renvoie 3 * 2 = 6
Facto(4) renvoie 4 * 6 = 24
```

Le traitement récursif s'achève lorsque toutes les valeurs sont retournées et que le résultat final est obtenu.

Etude d'un exemple : la fonction factorielle

Remarques

- ✓ La programmation récursive est une méthode très pratique pour résoudre certains types de problèmes. Elle offre une solution élégante et concise, permettant au programmeur d'obtenir un résultat correct avec très peu d'instructions.
- ✓ Tout problème exprimé de manière récursive peut également être reformulé sous une forme itérative, c'est-à-dire à l'aide de boucles.

Performance de la récursivité

- Les problèmes de mémoire et de rapidité sont les critères qui permettent de faire le choix entre la solution itérative et la solution récursive
- Pour le calcul de la fonction factorielle, les différences entre les deux solutions ne se voient pas trop. Pour l'utilisation de la mémoire, il y aura dépassement dans les calculs avant la saturation de la mémoire
- Autre exemple : Le calcul de la suite de Fibonacci

La suite de Fibonacci

- La relation de récurrence est définie par :

$$f_0 = 1$$

$$f_1 = 1$$

$$f_n = f_{n-2} + f_{n-1} \text{ pour } n > 1$$

- La solution récursive vient immédiatement de cette relation :

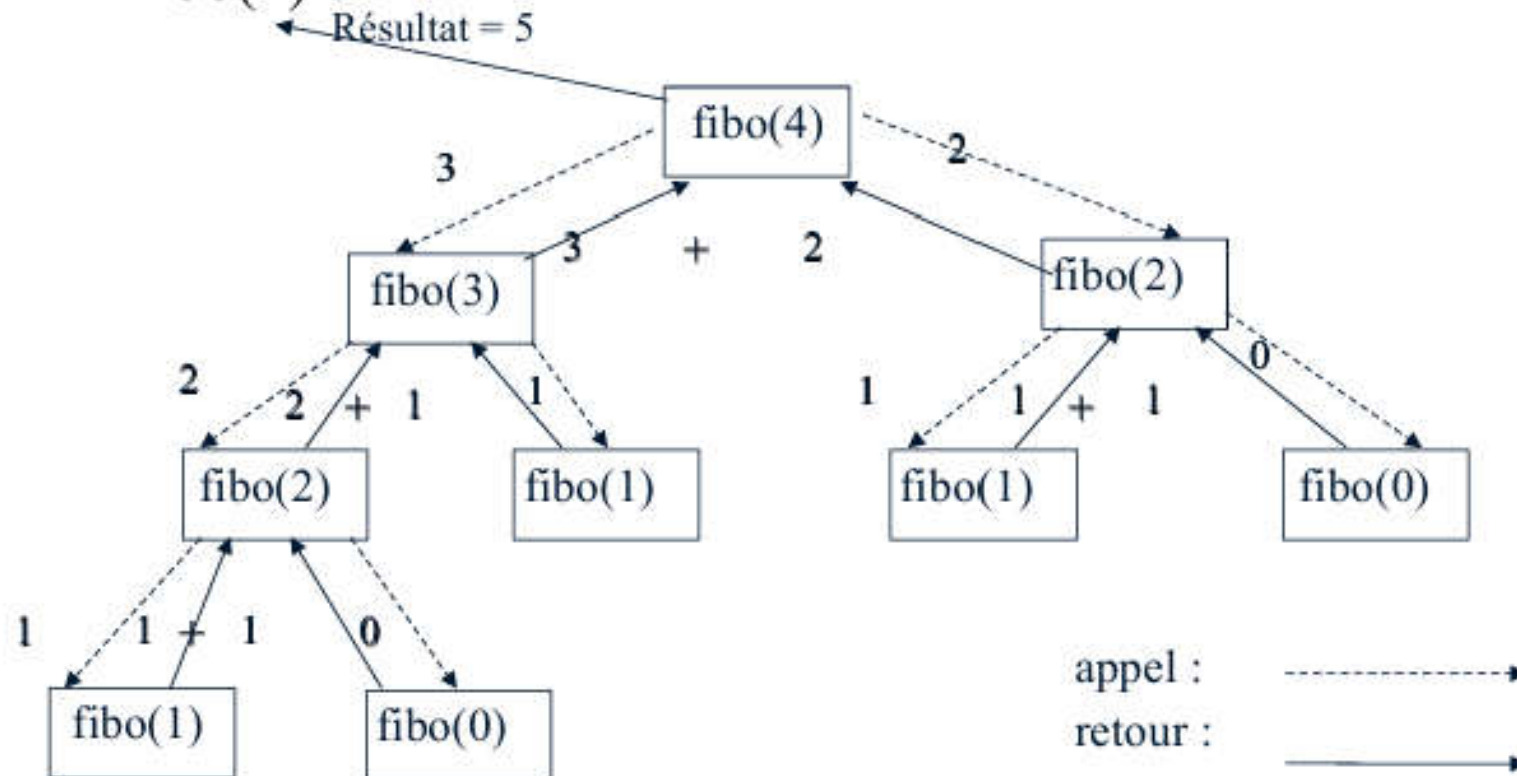
```
Fonction Fibo( n : Entier) : Entier
Début
Si (n = 0) ou (n = 1) Alors
  Fibo ← 1
Sinon
  Fibo ← Fibo(n - 2) + Fibo(n - 1)
FinSi
Fin
```

- On remarque que la fonction effectue deux appels, donc pour les valeurs importantes de n on aura des temps d'exécution importants (attente du résultat)

La suite de Fibonacci

A titre d'exemple, voici le schéma de calcul de Fibo(4)

- `fibo(4)` ?



La suite de Fibonacci

$$\begin{aligned} &= \text{Fibo}(3) + \text{Fibo}(2) \\ &= (\text{Fibo}(2) + \text{Fibo}(1)) + \text{Fibo}(2) \\ &= ((\text{Fibo}(1) + \text{Fibo}(0)) + \text{Fibo}(1)) + \text{Fibo}(2) \\ &= ((1 + \text{Fibo}(0)) + \text{Fibo}(1)) + \text{Fibo}(2) \\ &= ((1 + 1) + \text{Fibo}(1)) + \text{Fibo}(2) \\ &= (2 + \text{Fibo}(1)) + \text{Fibo}(2) \\ &= (2 + 1) + \text{Fibo}(2) \\ &= 3 + \text{Fibo}(2) \\ &= 3 + (\text{Fibo}(1) + \text{Fibo}(0)) \\ &= 3 + (1 + \text{Fibo}(0)) \\ &= 3 + (1 + 1) \\ &= 3 + 2 \\ &= \mathbf{5} \end{aligned}$$

Le nombre d'appels récursifs est donc très élevé (9 pour Fibo(4), 15 pour Fibo(5), 25 pour Fibo(6) et 21891 pour Fibo(20)), d'autant plus qu'il existe une méthode itérative simple pour calculer simultanément Fibo(n) et Fibo(n-1)

La solution itérative

```
Fonction Fibo(n : Entier) : Entier
Variables
i, fn, fn_0, fn_1 : Entier
Début
fn_1 ← 1
fn_2 ← 1
Pour i de 2 à n Faire
  fn ← fn_1 + fn_2
  fn_2 ← fn_1
  fn_1 ← fn
FinPour
Fibo ← fn
Fin
```

Choix entre itération et récursivité

- Le choix de la solution itérative peut être imposé par le **langage de programmation**. En effet, certains langages tels que Cobol n'est pas récursif
- D'autre part, seuls les problèmes dont la solution se base sur une **relation de récurrence avec une condition de sortie** peuvent être résolus de façon récursive
- Outre ces deux contraintes, le choix doit être fait uniquement en fonction des **critères d'efficacité (contraintes de temps et d'espace)**. Si la solution récursive satisfait ces critères, il n'y a pas lieu de chercher systématiquement une solution itérative

Choix entre itération et récursivité

Itération

```
procédure Itération( )  
  tantque (condition) faire  
    <Instructions>  
  fin tantque
```

```
fonction S(n:entier) : entier  
  s ← 0  
  tant que (n>0) faire  
    s ← s + n  
    n ← n - 1  
  fin tant que  
  retourner s
```

Récursivité

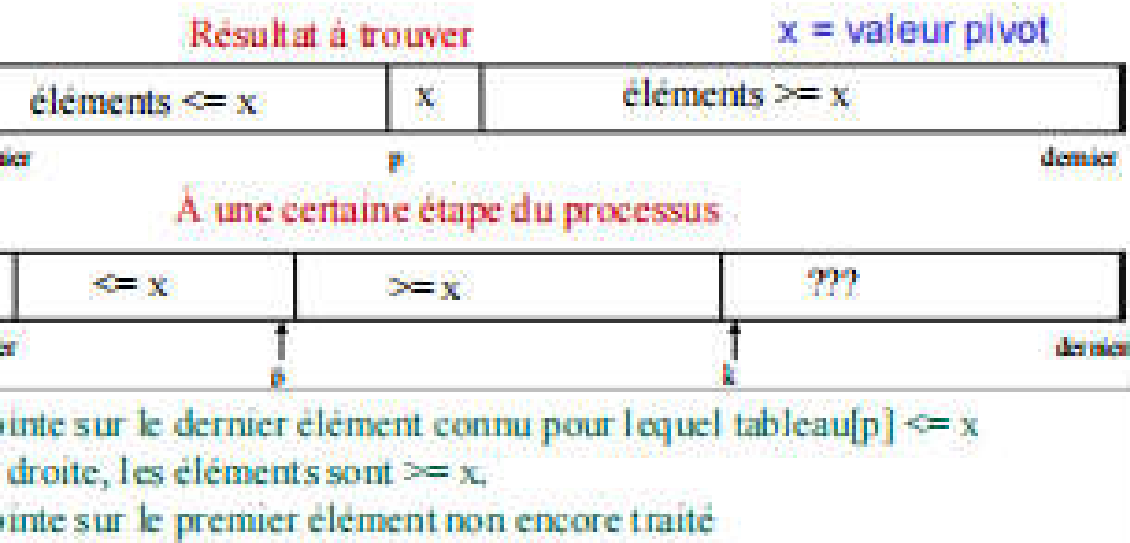
```
procédure Itération( )  
  si (condition) alors  
    <Instructions>  
    Itération()  
  finsi
```

```
fonction S(n:entier) : entier  
  Si (n=0) alors  
    retourner 0  
  sinon retourner S(n-1)+n  
  fin si
```

Le tri rapide (QuickSort)

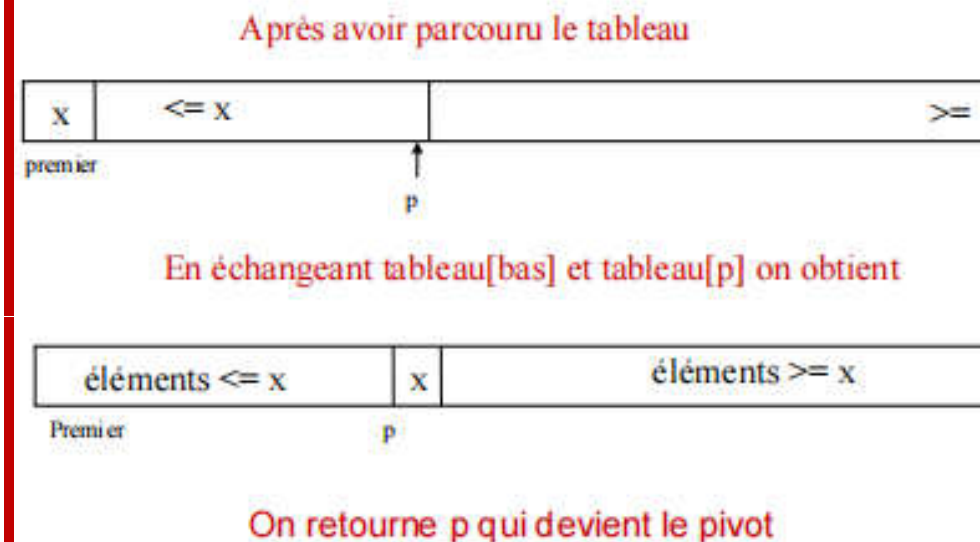
- tri récursif comportant 3 étapes
 - Étape 1: Partition autour du pivot
 - Un élément pivot du tableau à trier est choisi
 - Le tableau est réorganisé afin que:
 - les éléments $\leq$ au pivot se retrouvent avant le pivot
 - Les éléments $>$ pivot se retrouvent après le pivot
 - Étape 2
 - Un appel récursif est demandé pour trier les éléments avant le pivot
 - Étape 3
 - Un appel récursif est demandé pour trier les éléments après le pivot

Le tri rapide (QuickSort)



```

si tableau[k] ≤ x
    p ← p+1
    échanger(tableau[k], tableau[p])
fin si
    
```



Le tri rapide (QuickSort)

Description

- Tri un tableau récursivement entre les positions bas et haut

Variables

- tableau : tableau à trier
- premier : première position de l'intervalle à trier
- dernier : dernière position de l'intervalle à trier
- pivot : position d'une valeur pivot autour de laquelle s'effectue le tri

Logique

1. tri(tableau, premier, dernier)
2. si (premier < dernier)
3. pivot ← trouvePivot(tableau, premier, dernier)
4. tri(tableau, premier, pivot-1)
5. tri(tableau, pivot+1, dernier)
6. finsi

Le tri rapide (QuickSort)

```
Fonction partition( Var T:tableau; premier, dernier: entier) :entier
/variable
compteur, pivot, i: entier
debut
compteur ← premier
pivot ← T[premier]
pour i=premier+1 à dernier faire
    si T[i]<pivot alors //si l'élément est inférieur au pivot
compteur ← compteur+1 //on incrémente le compteur (modification de la
place finale du pivot)
    echanger(T,compteur,i) //on place l'élément à la position finale du pivot
    fin si
inpour
    echanger( T,compteur,premier) //on place le pivot à sa place
    retourner(compteur) //on renvoie la position finale du pivot
fin
```

Le tri rapide (QuickSort)

Procédure tri_rapide_bis(Var T:tableau d'entier, premier, dernier: entier)

Variable

pivot: entier

debut

 si premier < dernier **alors** *//condition d'arrêt de la récursivité*
 pivot ← partition(T,premier,dernier) *//partition du tableau en 2*
 tri_rapide_bis(T,premier,pivot-1) *//tri de la première partition*
 tri_rapide_bis(T,pivot+1,dernier) *//tri de la seconde partition*

finsi

fin

Procédure tri_rapide(T:Tableau d'entier)

debut

 tri_rapide_bis(T,1,taille(T)) *//tri du tableau entier*

fin

Exercice-1

Ecrire une fonction récursive PGCD_Euc qui retourne le PGCD de 2 entiers A et B en utilisant l'algorithme d'Euclide qui s'appuie sur les propriétés suivantes :

$\text{PGCD}(A,B) = B$ Si B est un diviseur de A

$\text{PGCD}(A,B) = \text{PGCD}(B, A \bmod B)$ Sinon

Exercices-1 correction

```
function PGCD_Euc(a,b : Entier) : Entier
begin
    if (a mod b = 0) alors
        PGCD_Euc ← b
    else
        PGCD_Euc ← PGCD_Euc(b,a mod b)
    fi
end
```

pgcd (a,b : entier) : entier

Début

Si (a=b) alors pgcd ← a

Sinon

Si (a>b) alors pgcd ← pgcd (a-b, b)

Sinon pgcd ← pgcd (b-a, a)

Finsi

Fin

exemple : pgcd (114, 42) =

pgcd (72, 42) =

pgcd (30,42) =

pgcd (12,30) =

pgcd (18,12) =

pgcd (6,12) =

pgcd (6,6) = 6

Exercice-2

- Ecrire un algorithme permettant de calculer la puissance d'un nombre réel x élevé à un entier naturel n

Exercices-2 correction

Fonction Puissance(x : réel, n : entier) : réel

Var

P : réel

Début

Si n = 0 alors

P ← 1

Sinon

Si n = 1 alors

P ← x

Sinon

P ← x * Puissance(x, n - 1)

FinSi

FinSi

Retourner P

Fin

Exemple Puissance(2, 3)

$2^3 = 2 \times \text{Puissance}(2, 2)$

Puis :

$2^2 = 2 \times \text{Puissance}(2, 1)$

Et :

$\text{Puissance}(2, 1) = 2$

Donc :

$\text{Puissance}(2, 1) = 2$

$\text{Puissance}(2, 2) = 2 \times 2 = 4$

$\text{Puissance}(2, 3) = 2 \times 4 = 8$

Exercice-3

- Ecrire deux fonctions qui calculent, de façon récursive, la somme de deux nombres a et b positifs ou nuls,

Exercices-3 correction

```
Fonction somme(a, b : entier) : entier
Début
    Si a = 0 alors
        somme ← b
    Sinon
        Si b = 0 alors
            somme ← a
        Sinon
            Si a > b alors
                somme ← 1 + somme(a, b - 1)
            Sinon
                somme ← 1 + somme(a - 1, b)
            FinSi
        FinSi
    FinSi
    Retourner somme
Fin
```

Exemple somme(3, 2) :

- $3 > 2 \rightarrow 1 + \text{somme}(3, 1)$
- $3 > 1 \rightarrow 1 + 1 + \text{somme}(3, 0)$
- $\text{somme}(3, 0) = 3$

Donc :

$\text{somme}(3,2)=1+1+3=5$